Stockd.

Software Design Description (SDD) — Architectural Design

CMSI 4071 — Senior Project I | Deliverable #4 (Architecture)

Team: Mason Lui, Leila Nawas, Paige Inoue

This document shows the current architecture for Stockd. It follows the same major components named in our SRS (Section 5.2). Some hardware details (final Pi camera mount and door sensor) may still change. The detailed design sections will be done later.

6.1 Introduction

Stockd is a smart fridge inventory system. A camera takes pictures of food going in or out of the fridge, and food recognition suggests what changed. Users review those guesses in a phone/web app, then the inventory updates. The app also shows expiration alerts and recipe ideas based on what is in the fridge. We are building this in stages, as described in our SDP: phone-camera first so we can test recognition and the app now, then Raspberry Pi + door sensor on a small demonstration fridge. Pantry tracking is not part of the required first version. This document explains the main software parts of Stockd and how they work together.

6.1.1 System Objectives

- Keep track of food in the fridge with less manual typing.
- Use a camera to help identify food being added or removed.
- Show food items, amounts, and expiration dates in an app.
- Remind users when food is close to expiring (in-app alerts).
- Suggest recipes using food the user already has.
- Let users add, remove, or fix food items manually.
- Let users review camera guesses before inventory is changed.

For our first version, we are focusing on the fridge. Pantry tracking may come later. Users may need to enter expiration dates for leftovers and other foods when the system cannot estimate them.

6.1.2 Hardware, Software, and Human Interfaces

6.1.2.1 Image Capture Interface (phone now, Pi later)

Today the main demo uses a phone camera in the Stockd web app (Scan for one photo, Live Track for motion → snapshot). We also have an OpenCV camera client that can use a laptop webcam or a still image. Later, a Raspberry Pi camera (and door-open/close sensor) will capture fridge images the same way and send them to the same backend. The final Pi mount and door wiring are still future hardware work.

6.1.2.2 Network Interface

The app and camera tools talk to our FastAPI backend over HTTP/HTTPS. In demos, a phone can reach the local app through a secure tunnel. The future Pi will use Wi-Fi to send pictures to that same backend API.

6.1.2.3 Food Recognition Interface

Recognition looks at a picture and returns candidate food labels (with confidence when available). Our running system uses OpenAI vision. If a key is missing, Stockd uses a clearly labeled mock recognizer so we can still demo. Wrong or unsure guesses are corrected in the review step before inventory is saved.

6.1.2.4 Data Management Interface

We store users, food items, amounts, expiration dates, scan sessions, and related history. Supabase (PostgreSQL + auth) is the intended persistent database from our SDP/SRS. For local demos we can also run mock mode with an in-memory store. The app reads and updates data through the backend API, not by talking to the database directly from every screen.

6.1.2.5 Application Interface (human / UI)

Users open Stockd in a phone, tablet, or laptop browser (React + Vite). They can log in, view and edit inventory, scan or live-track food, review recognition results, see expiration alerts, and browse recipe suggestions. The app can be added to a phone home screen as a simple web app (PWA).

6.1.2.6 Notification and Recipe Interfaces

Notification: checks stored dates and shows in-app expiration alerts (required baseline from our SRS). Operating-system push notifications are optional and not required for acceptance.

Recipes: look at current inventory and suggest meals, including which ingredients you have vs still need. Recipes do not remove food from inventory automatically.

6.2 Architectural Design

Stockd is split into the same major CSCs named in the SRS. That makes it easier to trace requirements to design and for teammates to own different areas.

High-level flow in the product we have now:

1. Capture an image (phone Scan/Live Track, or OpenCV client).
2. Send it to the backend for recognition.
3. Compare results to current inventory (what entered / left).
4. Let the user review and confirm.
5. Save inventory changes.
6. Use that inventory for alerts and recipes.

Phone capture and the future Pi client both call the same analyze/confirm APIs. Hardware work should not force a second inventory system.

6.2.1 Major Software Components

These match the SRS CSC breakdown, described as implemented in our codebase:

1. Image Capture CSC: Gets fridge/food images. Implemented as phone browser capture (Scan + Live Track) and the OpenCV camera client (hardware/raspberry_pi). Future Pi + door sensor will plug into this same role.

2. Food Recognition CSC: Turns an image into candidate food labels. Implemented in the backend recognition provider (OpenAI vision, optional Jev, or mock).

3. Inventory Management CSC: Creates, edits, removes, and lists food items. Also handles automatic add/remove proposals after recognition (reconciliation: entered vs left vs still there).

4. Expiration Management CSC: Stores and uses expiration / consume-by dates, and supports user edits when dates are unknown or wrong.

5. Application Interface CSC: The React web app screens: login, dashboard, inventory, scan, live track, review/confirm UI, alerts, recipes, settings.

6. Data Management CSC: Persistence and validation through the FastAPI backend + Supabase (or mock store in demo mode).

7. Notification CSC: Builds and shows expiration alerts in the app (read/unread, dismiss).

8. Recipe Suggestions CSC: Rule-based recipes matched to current inventory (have vs need).

Supporting runtime pieces (how the CSCs are hosted):

- Backend API (FastAPI): connects the app/camera clients to recognition, inventory, data, alerts, and recipes.
- Frontend host (Vite): serves the Application Interface CSC during development and demos.

6.2.2 Major Software Interactions

1. Image capture → recognition: The phone or camera client captures an image and sends it to POST /api/recognition/analyze. Live Track watches for motion, waits briefly for the scene to settle, then uses that same analyze call.

2. Recognition → inventory proposals: The backend runs food recognition, compares labels to the current inventory snapshot, and returns suggested additions and removals for review.

3. Review and correction: The Application Interface shows candidates. The user can accept, reject, or fix labels. Inventory is not changed until confirm (auto-confirm exists for demos but is optional).

4. Confirm → data management: POST /api/recognition/confirm writes accepted changes through Inventory Management into Data Management (Supabase or mock store). The app then shows the updated list.

5. Manual inventory edits: Users can still add/edit/remove items by hand if the camera misses something.

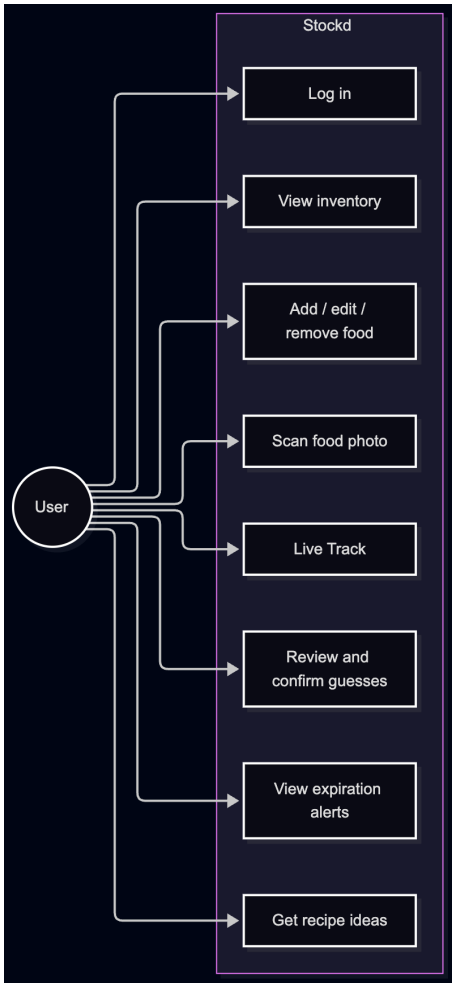
6. Expiration → notification: Notification checks dates on active items and shows in-app alerts.

7. Inventory → recipes: Recipe Suggestions reads current inventory and returns matching meal ideas without deleting ingredients.

6.2.3 Architectural Design Diagrams

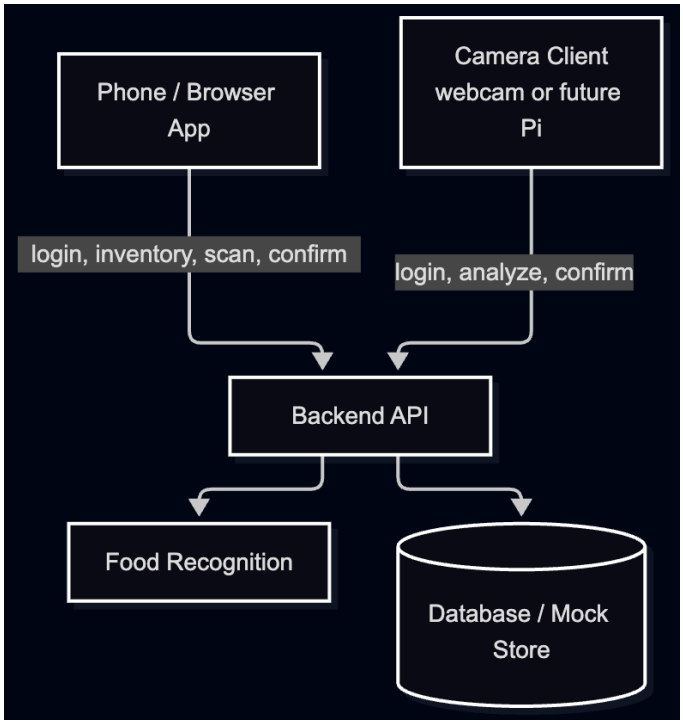
These diagrams show what users can do, how the main parts connect, and how a scan flows through the system. They focus on overall design rather than code details. Replace each placeholder below with the corresponding figure image.

Figure 1. Stockd use case diagram — main user actions



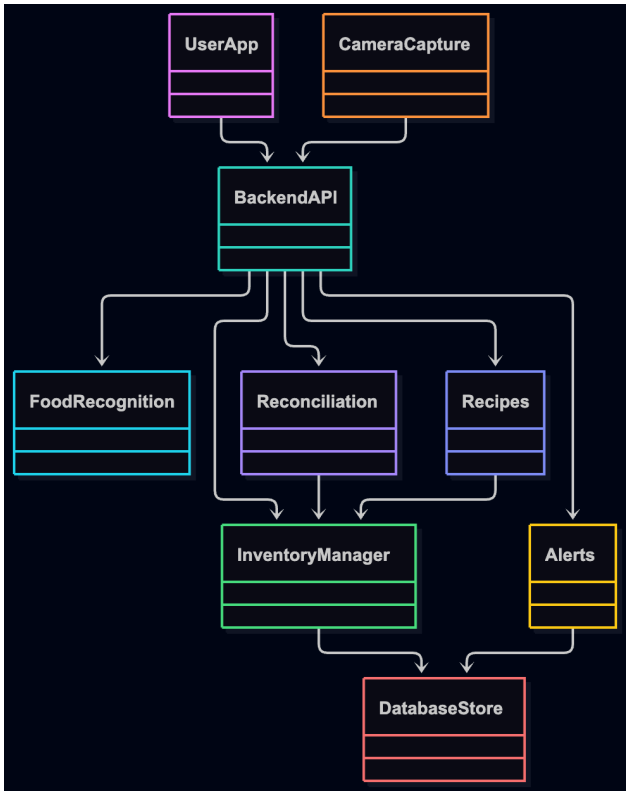
Caption: Stockd use case diagram — main user actions

Figure 2. Stockd component diagram — major services



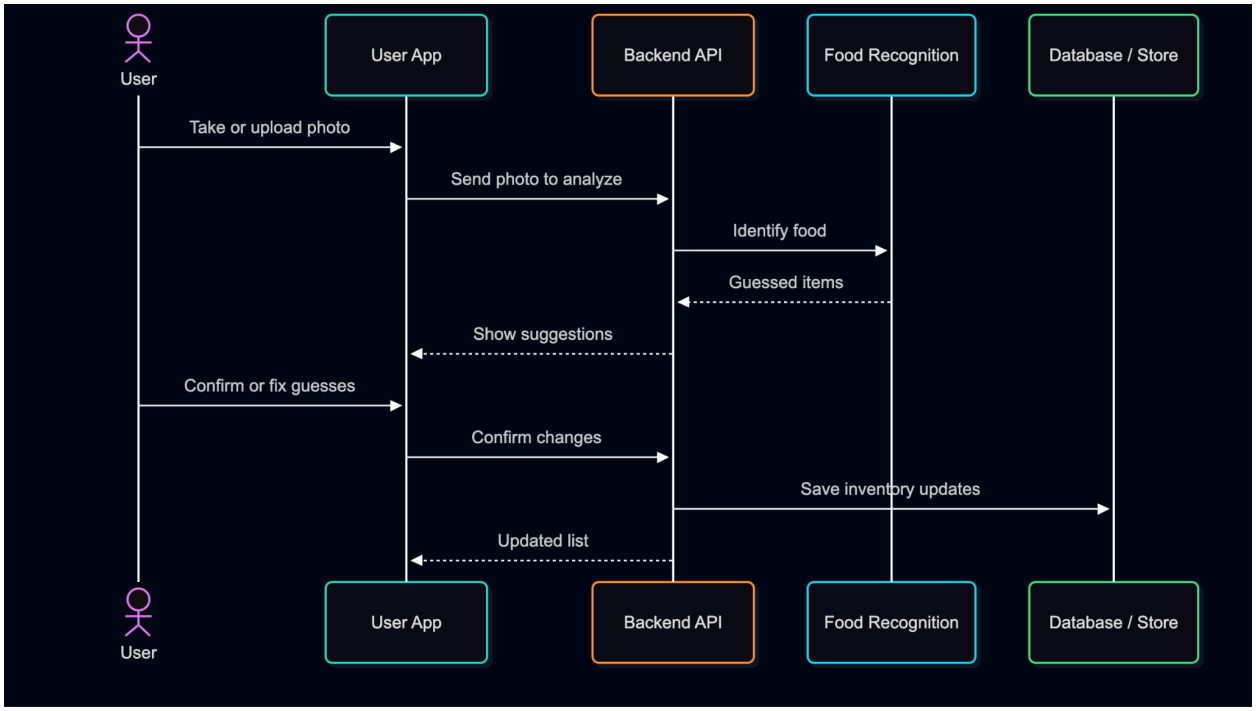
Caption: Stockd component diagram — major services

Figure 3. Stockd high-level module diagram



Caption: Stockd high-level module diagram

Figure 4. Basic flow — picture to inventory update



Caption: Basic flow — picture to inventory update

Architecture Notes and Open Decisions

Already true in our product / plan

- Phone-first capture is working (Scan + Live Track); Pi is next, same APIs
- Stack: React + Vite frontend, FastAPI backend, Supabase intended for persistence, OpenAI vision (mock for demos)
- Users review recognition before inventory writes
- In-app expiration alerts and recipe suggestions are baseline; OS push is not required
- Pantry tracking is out of baseline scope (same as SRS/SDP)

Still open / future hardware

- Final Raspberry Pi camera mount in the demonstration fridge
- Magnetic door sensor wiring vs motion-only trigger on the Pi
- Turning mock mode off for a shared live Supabase demo

This is the architecture part of the SDD for Deliverable #4. The detailed design sections (CSC/CSU class details and database design) will be done later and should continue using these same CSC names from the SRS.